

Git Rebase vs. Merge

1. How does git rebase work, and when is it preferable over merge?

A: The **git rebase** command takes the sequence of commits from your current branch, copies them, and reapplies them sequentially on top of the target branch's latest commit. It is preferable over merge when you want to maintain a **clean, linear history** without extra "merge commit" nodes. Rebase is often used to keep feature branches up-to-date with main before the final merge.

2. How do you update your feature branch with the latest changes from main?

A: The best way to update your feature branch is by using **git rebase main** (while you are on your feature branch). This action rewrites your branch's history, placing your feature commits after the most recent commits from the main branch. This results in a cleaner history than a standard merge.

3. How do you handle merge conflicts when the deadline is tight?

A: When the deadline is tight, the goal is speed and communication. I would immediately use a **powerful diff tool** (like VS Code or IntelliJ's built-in tools) to visualize the conflict and quickly accept the correct changes. If the conflict is complex, I would immediately pause, talk to the other developer involved, and jointly decide on the final structure.

4. How do you bring changes from another developer's branch into your own branch?

A: You first ensure you have the latest changes from the remote using **git fetch**. Then, you use **git merge <remote_name>/<branch_name>** (e.g., **git merge origin/teammate-feature**) or **git rebase <remote_name>/<branch_name>** to integrate those changes into your current local branch.

5. How do you keep track of updates made by teammates in a shared repository?

A: I keep track using **git fetch** followed by **git status** to see which remote branches have new commits. I also frequently use **git log origin/main** to review the commit messages and history of the main branch to understand the latest changes that have been integrated.

Branching Strategies

6. Can you explain GitFlow and GitHub Flow in simple terms?

A: **GitFlow** is a **strict** model for complex projects with formal releases; it uses two long-lived main branches (master for releases and develop for integration) plus dedicated branches for everything else. **GitHub Flow** is a **simple** model for continuous delivery; it uses only one long-lived branch (main), with all new features developed on short-lived feature branches that merge directly back into main.

7. Which branching strategy do you prefer for team projects, and why?

A: I prefer **GitHub Flow** for most modern team projects because it is simpler, faster, and better suited for continuous integration and continuous delivery (CI/CD). Its minimal set of rules reduces overhead and confusion, allowing the team to merge small changes frequently and ship products faster.

History Management & Cleanup

8. How do you completely remove the last commit from history using a hard reset?

A: You completely remove the last commit by using the command **git reset --hard HEAD~1**. This command moves the current branch pointer back one commit, and the **--hard** flag ensures that the files/changes associated with that last commit are **permanently discarded** from your working directory and local history.

9. What should you do if you accidentally commit sensitive information like passwords?

A: First, immediately change the exposed password or key, as the information is now public. To fix the history, you use a utility like **git filter-repo** to remove the sensitive data from **all prior commits**. Finally, you must use a **forced push** (git push --force) to overwrite the remote history with the cleaned version.

10. How do you restore an older version of a file without losing your current changes?

A: You restore an older version of a specific file by using the command **git checkout <commit_hash> -- path/to/file**. This pulls the file's content from the specified older commit and places it into your working directory without affecting any other files or discarding your uncommitted changes.

JAVA Interview Buddy